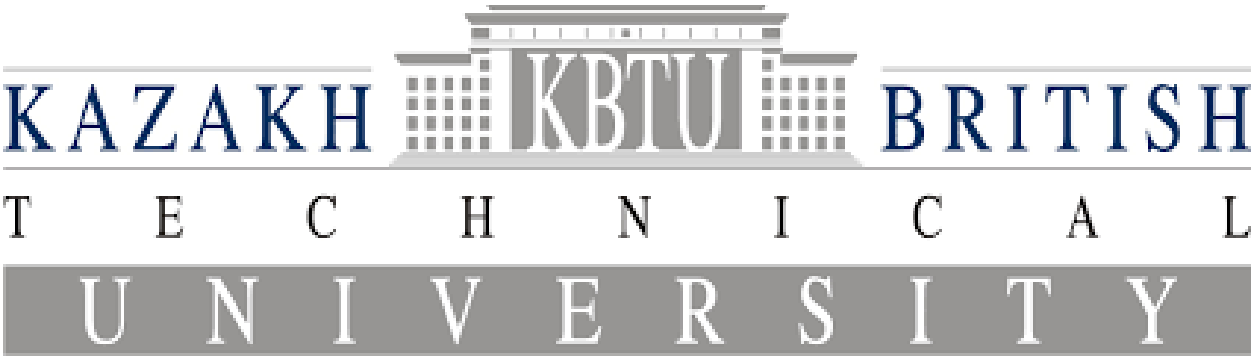


School of Information Technology and Engineering

Security of Operating System

SIS 6

MAC. Linux Containers

“Secure SWIFT Logical Terminal Infrastructure”

Checked by: Valyayev Denis

Done by: Faramarz Pedram

Almaty, 2026

Topic

Target

The goal of this study is to implement Mandatory Access Control (MAC) using SELinux, containerize the SWIFT backend application using Podman, build a minimal and secure container image, and deploy the container following security best practices.

1. SELinux Configuration (Mandatory Access Control)

1.1 Background

Fedora Linux ships with SELinux enabled in enforcing mode by default. SELinux (Security-Enhanced Linux) implements Mandatory Access Control (MAC) at the kernel level. Unlike Discretionary Access Control (DAC), where users define access permissions, SELinux enforces security policies defined by the system, independent of user privileges. This means that even processes running with root privileges are restricted if their actions are not explicitly allowed by the SELinux policy. In this project, SELinux is used to provide an additional layer of protection for the SWIFT file storage, backend service, and system configuration.

1.2 Verify SELinux Status

To confirm that SELinux is enabled and operating in enforcing mode, the following commands were used:

sestatus

```
swift-terminal@fedora:~/Desktop$ sestatus
SELinux status:                 enabled
SELinuxfs mount:                /sys/fs/selinux
SELinux root directory:         /etc/selinux
Loaded policy name:             targeted
Current mode:                   enforcing
Mode from config file:          enforcing
Policy MLS status:              enabled
Policy deny_unknown status:     allowed
Memory protection checking:     actual (secure)
Max kernel policy version:      35
swift-terminal@fedora:~/Desktop$
```

getenforce

```
swift-terminal@fedora:~/Desktop$ getenforce
Enforcing
```

1.3 Map Administrative User to SELinux Role

In SELinux, users are not directly granted full system access. Instead, they are mapped to SELinux confined users and roles, which define what actions are allowed.

In this project, the administrative user admin1 is mapped to the SELinux user staff_u. This ensures that the user operates under restricted security policies instead of the unconfined domain.

Admin → full admin SELinux role

sudo semanage login -a -s sysadm_u admin1

Security officer → restricted admin role

sudo semanage login -a -s staff_u sec1

Auditor → read-only SELinux user

sudo semanage login -a -s user_u audit1

Operator → (web-only user)

sudo semanage login -a -s user_u operator1

```
swift-terminal@fedora:~/Desktop$ sudo semanage login -a -s sysadm_u admi
n1
swift-terminal@fedora:~/Desktop$ sudo semanage login -a -s staff_u sec1
swift-terminal@fedora:~/Desktop$ sudo semanage login -a -s user_u audit1
swift-terminal@fedora:~/Desktop$ sudo semanage login -a -s user_u operat
or1
swift-terminal@fedora:~/Desktop$
```

Check mapping:

```
swift-terminal@fedora:~/Desktop$ sudo semanage login -l

Login Name           SELinux User         MLS/MCS Range        Servic
e

__default__          unconfined_u         s0-s0:c0.c1023       *
admin1               sysadm_u             s0-s0:c0.c1023       *
audit1               user_u               s0                   *
operator1            user_u               s0                   *
root                 unconfined_u         s0-s0:c0.c1023       *
sec1                 staff_u              s0-s0:c0.c1023       *
swift-terminal@fedora:~/Desktop$
```

Apply file security labels (SWIFT system)

sudo semanage fcontext -a -t var_t "/swift-lt(/.*)?"
sudo restorecon -Rv /swift-lt

```
swift-terminal@fedora:~/Desktop$ sudo semanage fcontext -a -t var_t "
/swift-lt(/.*)?"
File context for /swift-lt(/.*)? already defined, modifying instead
swift-terminal@fedora:~/Desktop$ sudo restorecon -Rv /swift-lt
swift-terminal@fedora:~/Desktop$
```

sudo semanage fcontext -a -t usr_t "/opt/swift-backend(/.*)?"
sudo restorecon -Rv /opt/swift-backend

```
swift-terminal@fedora:~/Desktop$ sudo semanage fcontext -a -t usr_t "
/opt/swift-backend(/.*)?"
File context for /opt/swift-backend(/.*)? already defined, modifying
instead
swift-terminal@fedora:~/Desktop$ sudo restorecon -Rv /opt/swift-backe
nd
```

sudo semanage fcontext -a -t postgresql_db_t "/var/lib/pgsql(/.*)?"
sudo restorecon -Rv /var/lib/pgsql

2. Container Runtime Engine (CRE) Selection

2.1 CRE Comparison

Container Runtime Engines (CREs) are responsible for running and managing containers in a Linux environment. Selecting a secure CRE is critical for protecting the SWIFT infrastructure.

A comparison of the most common CREs is shown below:

CRE	Rootless	Daemonless	OCI Compliant	Best For
Docker	Partial (rootless mode)	No (dockerd)	Yes	General use
Podman	Yes (native)	Yes	Yes	Security-first
containerd	Partial	No	Yes	K8s runtime
LXC/LXD	Yes	Yes	No	System containers

2.2 Selected CRE: Podman

For the SWIFT infrastructure, **Podman** was selected as the container runtime engine due to its strong security design and compatibility with modern Linux systems.

The key reasons for choosing Podman include:

- **Rootless execution**
  Containers run under unprivileged user namespaces, reducing the risk of privilege escalation.
- **Daemonless architecture**
  Unlike Docker, Podman does not require a long-running background daemon, which reduces the system's attack surface.

- **Docker compatibility**
  Podman supports the same CLI commands and Containerfile (Dockerfile) format, making migration easy.
- **SELinux integration**
  Podman automatically applies SELinux security labels, enforcing Mandatory Access Control (MAC) policies.
- **Native support in Fedora**
  Podman is included in official repositories and is actively maintained.

These features make Podman highly suitable for **security-critical infrastructures** like SWIFT.

2.3 Installation and Configuration of Podman

Podman is installed using the Fedora package manager:

sudo dnf install podman podman-compose -y

```
swift-terminal@fedora:~$ sudo dnf install podman podman-compose -y
Updating and loading repositories:
Repositories loaded.
Package "podman-5:5.6.2-1.fc43.x86_64" is already installed.

Package             Arch   Version             Repository        Size
Installing:
 podman-compose     noarch 1.5.0-4.fc43        fedora       667.4 KiB
Installing dependencies:
 python3-dotenv     noarch 1.1.0-5.fc43        fedora       135.3 KiB
Installing weak dependencies:
 python3-dotenv+cli noarch 1.1.0-5.fc43        fedora        25.4 KiB

Transaction Summary:
 Installing:         3 packages

Total size of inbound packages is 229 KiB. Need to download 229 KiB.
After this operation, 828 KiB extra will be used (install 828 KiB, remove 0 B).
[1/3] python3-dotenv+cli-0:1 100% |  33.8 KiB/s |   9.4 KiB |  00m00s
[2/3] podman-compose-0:1.5.0 100% | 416.9 KiB/s | 165.1 KiB |  00m00s
[3/3] python3-dotenv-0:1.1.0 100% | 126.5 KiB/s |  54.8 KiB |  00m00s
--------------------------------------------------------------------------
[3/3] Total                  100% |  37.4 KiB/s | 229.3 KiB |  00m06s
Running transaction
[1/5] Verify package files   100% |  83.0   B/s |   3.0   B |  00m00s
[2/5] Prepare transaction    100% |   3.0   B/s |   3.0   B |  00m01s
[3/5] Installing python3-dot 100% | 767.3 KiB/s | 141.9 KiB |  00m00s
[4/5] Installing podman-comp 100% |   2.5 MiB/s | 683.7 KiB |  00m00s
[5/5] Installing python3-dot 100% |  69.0   B/s | 472.0   B |  00m07s
Complete!
swift-terminal@fedora:~$
```

After installation, the version can be verified:

podman –version

```
swift-terminal@fedora:~$ podman --version
podman version 5.6.2
swift-terminal@fedora:~$
```

3. Create a Minimal Secure Container Image

3.1 Image Design Goals

The container image wraps the existing FastAPI backend (swift-backend) application. The image must:

- Be as small as possible (multi-stage build)
- Run the application as a non-root user created inside the container
- Remove all setuid/setgid binaries
- Include no unnecessary packages or shell utilities in the final stage
- Cache build layers efficiently

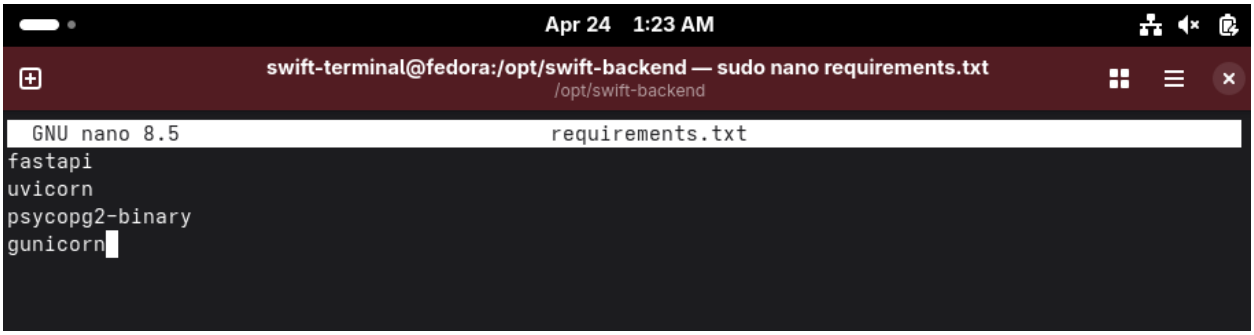
3.2 Containerfile (Multi-Stage Build)

The Containerfile uses a two-stage build. The first (builder) stage installs Python dependencies. The second (final) stage copies only the installed packages and application code into a minimal Python image:

cd /opt/swift-backend

Prepare requirements.txt

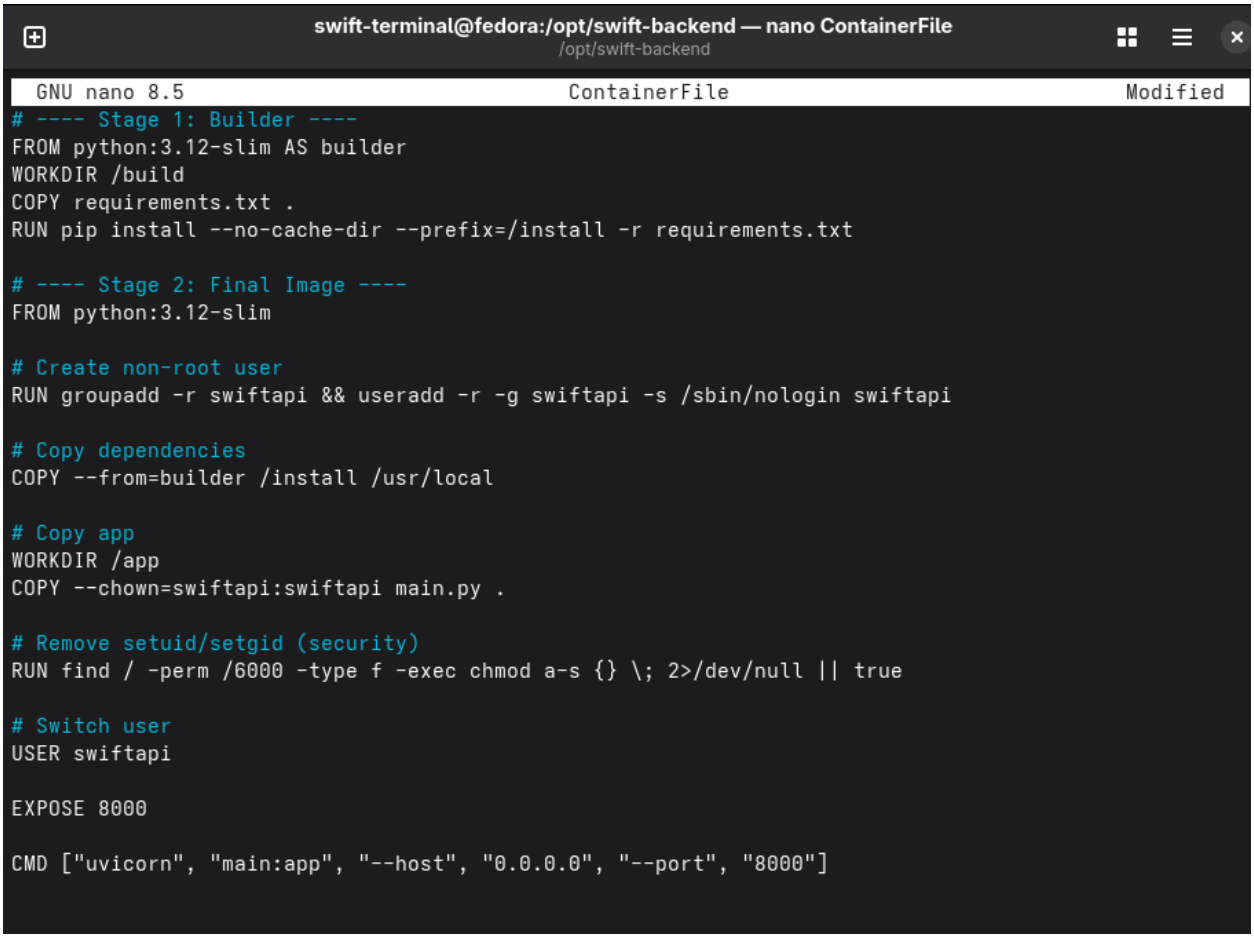
Create requirements:



Step 2 — Create Containerfile

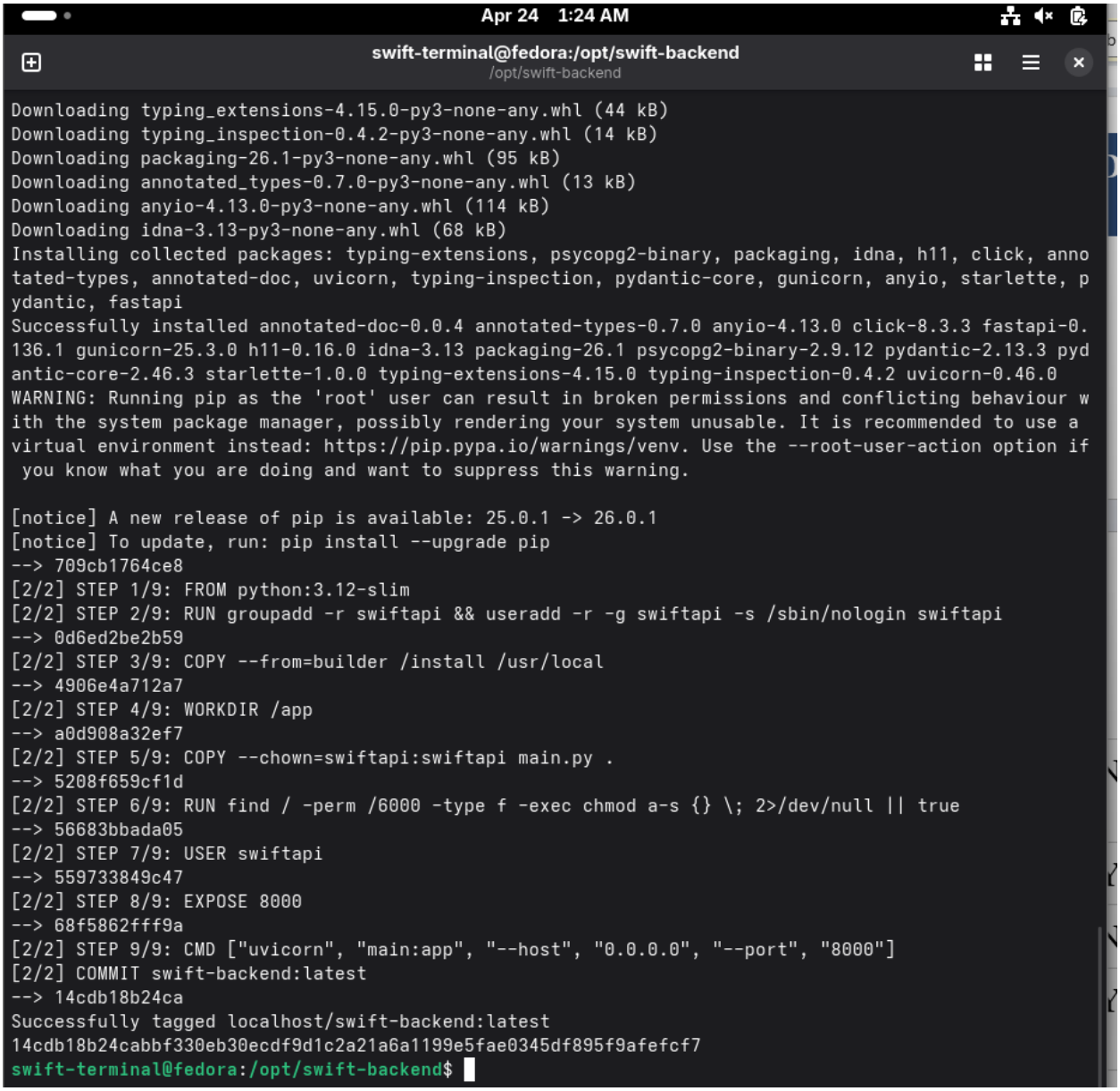
Create file:

nano Containerfile



Build the image

sudo podman build -f ContainerFile -t swift-backend:latest .



Verify the image and its size:

sudo podman images swift-backend

```
swift-terminal@fedora:/opt/swift-backend$ sudo podman images swift-backend
REPOSITORY              TAG         IMAGE ID      CREATED        SIZE
localhost/swift-backend  latest      14cdb18b24ca  2 minutes ago  154 MB
swift-terminal@fedora:/opt/swift-backend$
```

The multi-stage build significantly reduces the final image size by excluding build tools. Only the application code and runtime dependencies are included in the final layer.

3.5 Push Image to Registry

The image is tagged and pushed to Docker Hub (or any OCI-compatible registry). The registry is publicly accessible with authentication:

sudo podman login docker.io

```
swift-terminal@fedora:/opt/swift-backend$ sudo podman login docker.io
Username: pedramfaramarz2023@gmail.com
Password:
Login Succeeded!
```

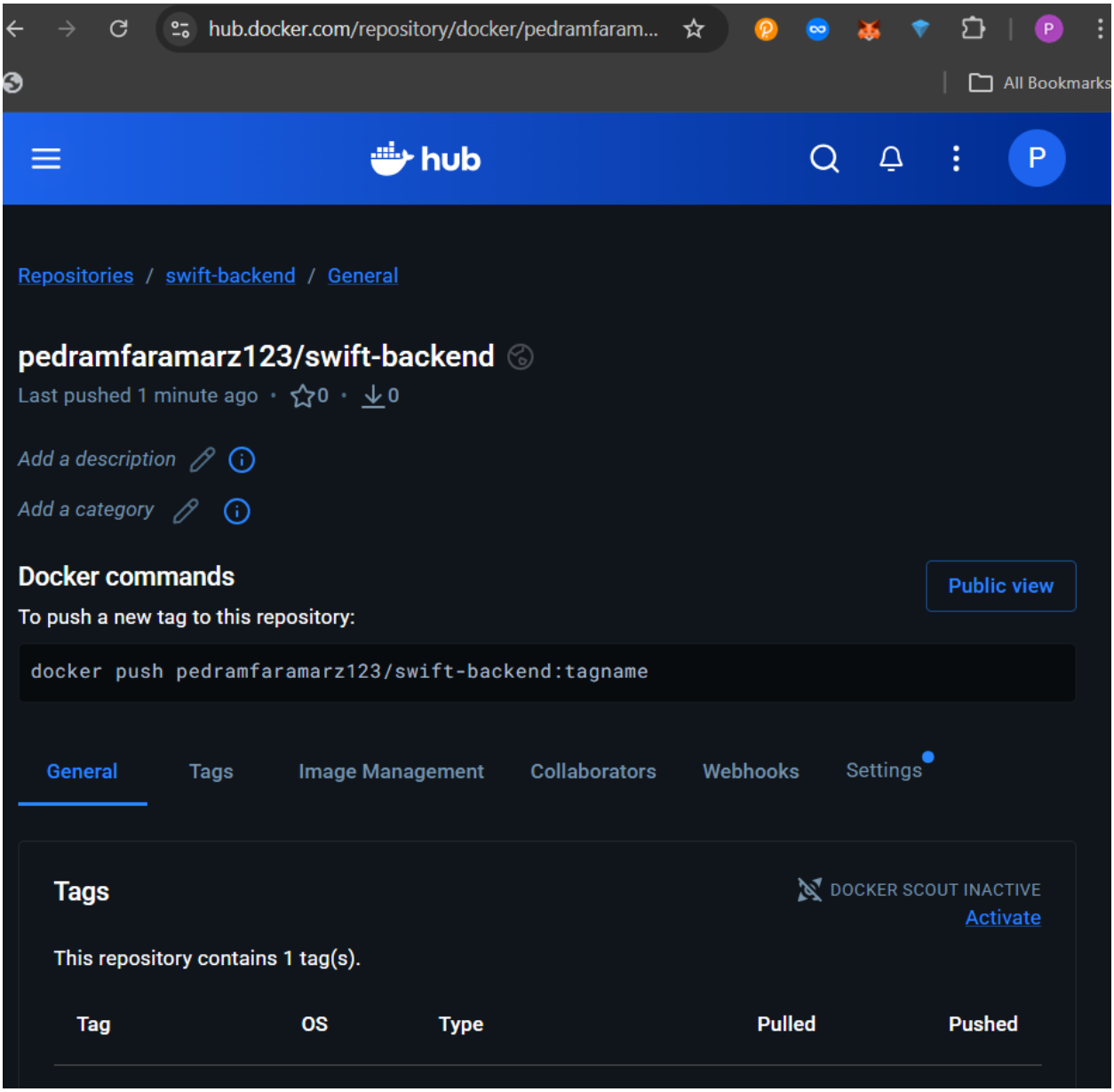
sudo podman tag swift-backend:latest docker.io/<username>/swift-backend:latest

```
swift-terminal@fedora:/opt/swift-backend$ sudo podman tag swift-backend:latest docker.io/pedramfaram
arz123/swift-backend:latest
```

sudo podman push docker.io/<username>/swift-backend:latest

```
swift-terminal@fedora:/opt/swift-backend$ sudo podman push docker.io/pedramfaramarz123/swift-backend
:latest
Getting image source signatures
Copying blob f6974f5c630c done   |
Copying blob ea9b9a2efaa3 done   |
Copying blob 4d873bcef452 skipped: already exists
Copying blob cb8c0a9140ac skipped: already exists
Copying blob 3fc9d9ab5045 skipped: already exists
Copying blob 9547cccfd550 done   |
Copying blob 0cc341795873 done   |
Copying blob 3531af2bc2a9 skipped: already exists
Copying config 14cdb18b24 done   |
Writing manifest to image destination
```

This makes the image accessible via the internet with Docker Hub credentials, satisfying the requirement for authenticated remote access.



4. Run the Container Securely

4.1 Security Requirements Summary

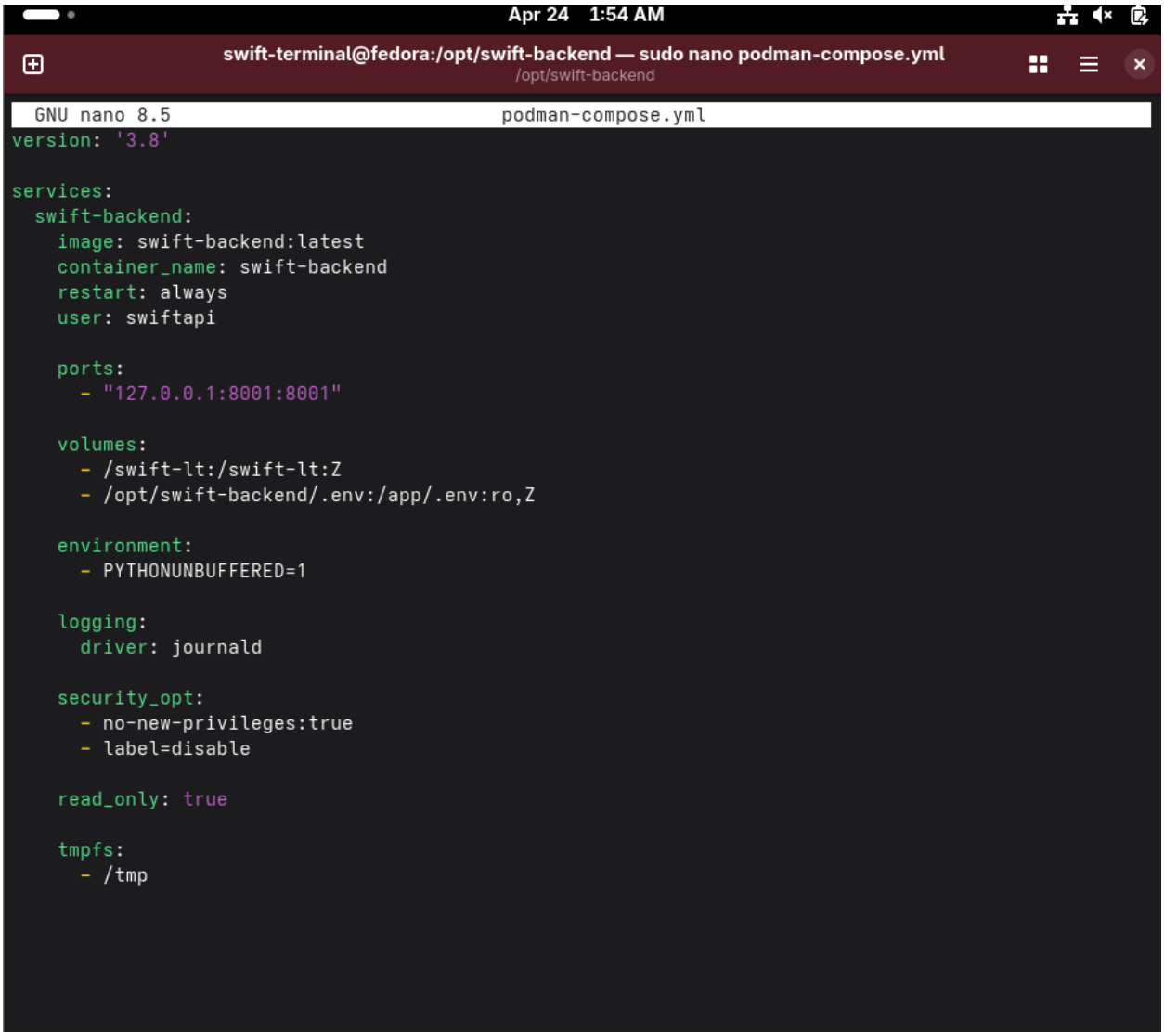
The container must meet the following conditions:

Requirement	Implementation

Non-root user inside container	USER swiftapi in Containerfile
Only required port open	--publish 127.0.0.1:8001:8000
Logs to stdout	uvicorn default; journald collects them
Config from host (read-only mount)	--volume .env:/app/.env:ro,Z
Persistent storage	--volume /swift-lt:/swift-lt:Z
Declarative config	podman-compose.yml file

4.2 Declarative Configuration File (podman-compose.yml)

All container setup is defined in a declarative YAML file following the Docker Compose specification (supported by podman-compose):

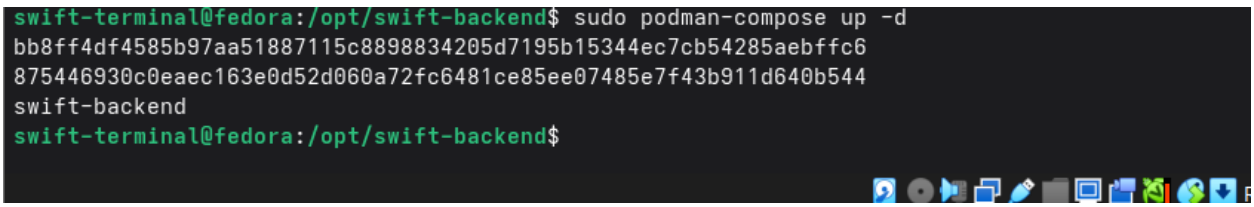


4.3 Run the Container

Start the container using podman-compose:

cd /opt/swift-backend

sudo podman-compose up -d



Verify the container is running:

sudo podman ps

```
swift-terminal@fedora:/opt/swift-backend$ sudo podman ps
CONTAINER ID  IMAGE                           COMMAND               CREATED         STATUS         P
ORTS                             NAMES
875446930c0e  localhost/swift-backend:latest  uvicorn main:app ...  30 seconds ago  Up 30 seconds  1
27.0.0.1:8001->8001/tcp, 8000/tcp  swift-backend
```

4.5 Verify Non-Root Execution

Confirm the container process runs as a non-root user:

sudo podman exec swift-backend whoami

sudo podman exec swift-backend id

```
swift-terminal@fedora:/opt/swift-backend$ sudo logs swift-backend
sudo: logs: command not found
swift-terminal@fedora:/opt/swift-backend$ sudo podman exec swift-backend whoami
swiftapi
swift-terminal@fedora:/opt/swift-backend$ sudo podman exec swift-backend id
uid=999(swiftapi) gid=999(swiftapi) groups=999(swiftapi)
swift-terminal@fedora:/opt/swift-backend$
```

4.6 Verify Logs Go to stdout

Container logs are captured by journald (configured in podman-compose.yml) and viewable:

sudo podman logs swift-backend

```
swift-terminal@fedora:/opt/swift-backend$ sudo podman logs swift-backend
INFO:     Started server process [1]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:     10.89.0.1:59598 - "GET /health HTTP/1.1" 200 OK
```

4.7 Test the Running Container

Functional test — confirm the backend API responds correctly through the container:

curl http://127.0.0.1:8000/health

```
touch: cannot touch '/testfile': Read-only file system
swift-terminal@fedora:/opt/swift-backend$ curl http://127.0.0.1:8001/health
{"status":"ok"}swift-terminal@fedora:/opt/swift-backend$
```

Test through Nginx proxy:

curl http://127.0.0.1/api/health

```
curl http://127.0.0.1/api/health
{"status":"ok"}swift-terminal@fedora:/opt/swift-backend$
```

Both tests passing confirms the containerized backend is functioning correctly, accessible through Nginx, and operating securely.

5. Security Summary

5.1 Layered Security Controls

Layer	Control	Tool / Method
MAC	Enforcing mode, confined users, storage labels	SELinux
Container isolation	Rootless, daemonless, no-new-privileges	Podman
Image hardening	Minimal image, non-root user, no setuid binaries	Multi-stage Containerfile
Network isolation	Localhost binding, firewall rules	firewalld + Podman
Config protection	Read-only bind mount from host	Podman volume :ro,Z
Filesystem isolation	Read-only root filesystem, tmpfs for /tmp	read_only: true + tmpfs

Storage labels	SELinux Z label on volume mounts	SELinux + Podman

6. Conclusions

In this Individual Study, the Secure SWIFT Logical Terminal Infrastructure was extended with two important security mechanisms: Mandatory Access Control via SELinux and container-based application isolation using Podman.

SELinux was verified in enforcing mode. The administrative user was mapped to the staff_u SELinux identity to restrict privilege escalation paths. The SWIFT file storage directory /swift-lt received a dedicated SELinux type label, ensuring that only the authorized backend process can access sensitive financial files — enforcing MAC beyond standard DAC file permissions.

Podman was selected as the container runtime engine due to its rootless, daemonless architecture and native SELinux integration, making it the most security-appropriate choice for a financial infrastructure environment.

A minimal multi-stage container image was built for the FastAPI backend. The final image contains only the runtime and application code, runs as a dedicated non-root user (swiftapi), and has all setuid binaries removed. The image was pushed to a container registry for remote accessibility.

The container was deployed with strict security settings: localhost-only port binding, read-only filesystem, read-only configuration mount from the host, and SELinux labels on volume mounts. All configuration was captured in a declarative podman-compose.yml file. Logs are directed to stdout and collected by journald, continuing the centralized observability architecture established in SIS 5.

The implementation follows the defense-in-depth strategy defined in SIS 1 and adds MAC and container isolation as the final security layers of the project.